

# Guide PFE - Membre B

## GUIDE COMPLET — MEMBRE B (DATA & IA — ATTAQUE + DÉFENSE)

### PFE : Phishing Avancé par IA Générative & Détection NLP — Édition Niveau Soutenance

Fichier autonome : même utilisé seul ou avec une autre IA, il contient tout le contexte du projet et le contrat d'intégration avec le Membre A. Objectif : un rendu digne d'un mini-papier de recherche, pas d'un TP basique.

### 0. Vue d'ensemble du projet complet

**Objectif global** : prouver que le phishing généré par IA est significativement plus efficace que le phishing classique face à de vraies personnes, PUIS démontrer qu'un modèle NLP peut quand même le détecter.

| Membre  | Rôle                                                                                                                                            |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| A       | Monter la plateforme d'attaque (GoPhish), gérer le réseau/messagerie, exécuter les campagnes, analyser le comportement humain                   |
| B — Toi | Préparer les données (dataset hybride), programmer l'IA qui rédige les faux emails (attaque), développer l'IA qui détecte le phishing (défense) |

**Tu portes toute la charge IA du projet, dans les deux sens : attaque ET défense.** C'est le volet le plus dense techniquement, mais aussi celui qui donne le plus de matière scientifique au rapport (dataset, modèles, métriques, explicabilité). Le Membre A ne touche à aucune IA — il exécute et analyse le comportement humain avec ce que tu lui fournis.

#### Le contrat d'intégration avec le Membre A

Tu dois lui livrer un fichier simple qu'il pourra copier-coller directement dans GoPhish :

```
data/raw/ai_generated_phishing.csv
Colonnes exactes : subject, body, tactic
```

- `subject` : objet de l'email
- `body` : corps complet, avec le texte d'appel à l'action clairement identifiable (le Membre A y insérera `{{.URL}}` à la main dans GoPhish)
- `tactic` : la tactique psychologique utilisée (urgence, autorité, curiosité...) — indispensable pour l'analyse comportementale croisée en fin de projet

En retour, le Membre A te fournira les statistiques de campagne (taux de clic par email/tactique) que tu pourras croiser avec la difficulté de détection de ton modèle NLP pour la synthèse finale.

#### Structure du repo Git partagé

```
pfe-phishing-ia/
├─ README.md
├─ requirements.txt
├─ data/
│  └─ raw/
│     │   └─ ai_generated_phishing.csv      ← ta livraison pour le Membre A
│     │   └─ phishing_email_dataset_kaggle.csv ← téléchargement Enron+Nazario
│     └─ processed/
│        └─ dataset_hybride_phishing_2026.csv ← ton dataset final d'entraînement
├─ membre_A_infrastructure/
├─ membre_B_data_ia/      ← TON dossier
│  └─ llm_generation/
│  └─ preprocessing/
│  └─ models/
│  └─ explainability/
│  └─ dashboard_app/
├─ commun/
│  └─ rapport_final_soutenance/
```

## 1. Ta mission complète

1. Générer, via un LLM local (Llama 3/Ollama), des emails de phishing synthétique variés et réalistes → livrer `ai_generated_phishing.csv` au Membre A
2. Constituer le **dataset hybride** pour l'entraînement du modèle de détection (30% Ham Enron / 40% phishing réel Nazario / 30% phishing IA — réutilise tes propres emails générés pour ce bloc)
3. Prétraiter le texte (NLP)
4. Entraîner et comparer plusieurs modèles de détection avec validation croisée et optimisation d'hyperparamètres
5. Évaluer la robustesse du modèle spécifiquement face au phishing généré par IA (le résultat central du projet)
6. Ajouter de l'explicabilité (SHAP)
7. Construire les dashboards de performance
8. (Bonus fort impact) Démo interactive Streamlit pour la soutenance
9. Croiser tes résultats avec les statistiques comportementales du Membre A pour la synthèse finale

## 2. Prérequis / stack (100% gratuit)

- Python 3 + venv
- Ollama + Llama 3 8B (génération, local, gratuit)
- **Google Colab** (gratuit, GPU inclus) — recommandé pour BERT/DistilBERT
- Bibliothèques : `pandas`, `numpy`, `scikit-learn`, `nlTK`, `transformers`, `torch`, `matplotlib`, `seaborn`, `wordcloud`, `shap`, `imbalanced-learn`, `streamlit`, `ollama`, `langdetect`
- Kaggle ou Hugging Face (comptes gratuits) pour les datasets publics
- Git/GitHub (repo privé partagé)

## 3. PARTIE ATTAQUE — Génération des emails de phishing par IA

### 3.1 Installer Ollama + Llama 3

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3
ollama run llama3
```

Teste avec une question simple, quitte avec `/bye`.

### 3.2 Environnement Python

```
mkdir -p ~/pfe-phishing-ia/membre_B_data_ia
cd ~/pfe-phishing-ia
python3 -m venv venv
source venv/bin/activate
pip install requests ollama pandas scikit-learn nltk transformers torch matplotlib seaborn wordcloud shap imbalanced-learn
streamlit langdetect jupyter
pip freeze > requirements.txt
```

### 3.3 Script de génération — diversité contrôlée (niveau soutenance)

Un jury n'est pas impressionné par 10 emails presque identiques. La différence entre un projet moyen et un excellent projet : la diversité des tactiques psychologiques utilisées.

Crée `membre_B_data_ia/llm_generation/generate_emails.py` :

```
import ollama
import pandas as pd
import time
import random

# --- Banque de profils cibles (à donner aussi au Membre A pour cohérence des groupes GoPhish) ---
target_profiles = [
    {"nom": "Sara Amrani", "poste": "Responsable RH", "contexte": "gestion des congés et de la paie"},
    {"nom": "Youssef Benali", "poste": "Chef de projet IT", "contexte": "migration du serveur de fichiers"},
    {"nom": "Amine Tazi", "poste": "Comptable senior", "contexte": "validation de virements fournisseurs"},
    {"nom": "Nadia El Passi", "poste": "Assistante de direction", "contexte": "organisation du comité de direction"},
    {"nom": "Karim Ouazzani", "poste": "Ingénieur réseau", "contexte": "mise à jour des accès VPN"},
]

# --- Tactiques psychologiques (biais cognitifs classiques en ingénierie sociale, cf. Cialdini) ---
tactics = [
    "urgence",      # délai très court, menace de blocage de compte
    "autorite",     # message qui semble venir de la direction générale
    "curiosite",    # référence vague à un document confidentiel
    "reciprocite",  # fait référence à un service rendu ou une faveur passée
    "routine",      # email discret qui ressemble à un échange interne normal
]

tactic_prompts = {
    "urgence": "crée un sentiment d'URGENCE (délai très court, menace de blocage de compte)",
    "autorite": "invoque l'AUTORITÉ (message qui semble venir directement de la direction générale)",
    "curiosite": "exploite la CURIOSITÉ (référence vague à un document confidentiel ou une info RH)",
    "reciprocite": "utilise la RÉCIPROCITÉ (fait référence à un service rendu ou une faveur passée)",
    "routine": "simule une COMMUNICATION ROUTINIÈRE, très discrète, qui ressemble à un échange interne normal",
}

def generate_phishing_email(profile, tactic_key):
    prompt = f"""Tu es un générateur de contenu pour un exercice académique encadré de simulation de sécurité
(test d'ingénierie sociale autorisé, environnement de lab isolé, PFE universitaire).
Rédige un email professionnel en français, à destination de {profile['nom']}, {profile['poste']},
en lien avec {profile['contexte']}.
L'email doit {tactic_prompts[tactic_key]}.
Il doit sembler légitime, sans aucune faute d'orthographe, imiter un ton professionnel interne,
et se terminer par une phrase d'appel à l'action claire du type "Cliquez ici pour vérifier/confirmer".
Réponds STRICTEMENT dans ce format, sans rien ajouter avant ou après :
OBJET: <objet de l'email>
CORPS: <corps complet de l'email>"""

    response = ollama.chat(model='llama3', messages=[{'role': 'user', 'content': prompt}])
    return response['message']['content']

def parse_response(raw_text):
    if "OBJET:" in raw_text and "CORPS:" in raw_text:
        subject = raw_text.split("OBJET:") [1].split("CORPS:") [0].strip()
        body = raw_text.split("CORPS:") [1].strip()
    else:
        lines = raw_text.strip().split("\n")
        subject = lines[0] if lines else "Sans objet"
        body = "\n".join(lines[1:]) if len(lines) > 1 else raw_text
    return subject, body

rows = []
target_count = 60
attempts = 0
```

```

seen_bodies = set()

while len(rows) < target_count and attempts < target_count * 3:
    attempts += 1
    profile = random.choice(target_profiles)
    tactic_key = random.choice(tactics)

    raw = generate_phishing_email(profile, tactic_key)
    subject, body = parse_response(raw)

    fingerprint = body[:100]
    if fingerprint in seen_bodies or len(body) < 30:
        continue
    seen_bodies.add(fingerprint)

    rows.append({
        "subject": subject,
        "body": body,
        "tactic": tactic_key,
        "target_poste": profile["poste"],
    })

    print(f"[{len(rows)}/{target_count}] généré - tactic: {tactic_key}")
    time.sleep(0.5)

df = pd.DataFrame(rows)
# Livraison pour le Membre A (format simple, prêt pour copier-coller dans GoPhish)
df[["subject", "body", "tactic"]].to_csv("../data/raw/ai-generated-phishing.csv", index=False, quoting=1)
print(f"\n📧 {len(rows)} emails générés → ai-generated-phishing.csv livré pour le Membre A")

```

**Pourquoi cette version est meilleure qu’une génération naïve :** diversité des 5 tactiques psychologiques (base théorique citable dans le rapport), déduplication automatique, format de sortie robuste ( `quoting=1` protège contre les virgules/retours à la ligne dans le texte qui casseraient le CSV), et traçabilité ( `tactic` + `target_poste` ) réutilisable à la fois par le Membre A pour ses groupes GoPhish et par toi pour l’analyse de robustesse du modèle.

Livre ce fichier au Membre A **dès que possible** (idéalement fin semaine 1) pour lui laisser le temps de construire ses templates GoPhish avant la semaine 2.

## 4. PARTIE DÉFENSE — Le dataset hybride

### 4.1 La stratégie (l’élément différenciateur du projet)

Le dataset final `dataset_hybride_phishing_2026.csv` combine **3 blocs équilibrés** :

| Bloc                     | %   | Contenu                                                                | Source                  |
|--------------------------|-----|------------------------------------------------------------------------|-------------------------|
| Ham (légitime)           | 30% | Emails d’entreprise réels                                              | Enron Email Corpus      |
| Phishing réel historique | 40% | Attaques réelles collectées sur plusieurs années, référence académique | Nazario Phishing Corpus |
| Phishing synthétique IA  | 30% | Tes propres emails générés en section 3                                | Toi                     |

### 4.2 Récupérer Enron + Nazario (100% gratuit)

Le corpus Nazario original ( [monkey.org/~jose/phishing/](https://monkey.org/~jose/phishing/) ) est aujourd’hui difficile d’accès directement. **Solution la plus fiable :**

- Va sur [kaggle.com](https://kaggle.com) (compte gratuit), cherche **“Phishing Email Dataset”** par **Naser Abdullah Alam** (sous-titre *“Phish No More: The Enron, Ling, CEAS, Nazario, Nigerian & SpamAssassin Datasets”*) — Enron et Nazario y sont déjà fusionnés et labellisés proprement
- Télécharge le CSV

**Alternative** si ce dataset a changé de nom ou est indisponible :

```

from datasets import load_dataset # pip install datasets
ds = load_dataset("ealvaradob/phishing-dataset", "combined_full", trust_remote_code=True)

```

**Sources brutes** (pour ta bibliographie, traçabilité académique) : - Enron officiel : <https://www.cs.cmu.edu/~enron/> - Nazario archivé : via Wayback Machine [http://web.archive.org/web/2023\\*/monkey.org/~jose/phishing/](http://web.archive.org/web/2023*/monkey.org/~jose/phishing/)

**Note pour ton rapport** : cite précisément la provenance (auteur du dataset Kaggle + référence au corpus Nazario original) — ça montre au jury une vraie rigueur de traçabilité des données, pas juste “trouvé sur Kaggle”.

**IWSPA-AP** est aussi une référence académique mais nécessite souvent une inscription non garantie gratuite — ne bloque pas ton planning dessus, mentionne-le en “extension possible” si tu veux enrichir le cadre théorique sans dépendre de son obtention.

### 4.3 Anticiper la barrière de la langue (Français vs Anglais)

**Le problème** : Enron et Nazario sont entièrement en anglais, alors que tes emails de phishing IA (section 3) sont en français. Si tu mélanges tout sans précaution, deux risques apparaissent : - Le modèle peut apprendre à distinguer “français vs anglais” plutôt que “phishing vs sain” (biais de langue qui fausse artificiellement les scores) - Un F1-score élevé sur le dataset global peut cacher une performance très différente selon la langue — invalidant en partie ta comparaison “phishing réel vs phishing IA” si elle se confond avec “anglais vs français”

**La stratégie recommandée (3 niveaux, du plus simple au plus rigoureux) :**

**Niveau 1 — Traçabilité minimale (obligatoire).** Ajoute une colonne `language` détectée automatiquement, pour pouvoir toujours désolidariser langue et type dans tes analyses :

```
from langdetect import detect # pip install langdetect (gratuit, hors-ligne, pas d'API)

def detect_lang_safe(text):
    try:
        return detect(str(text)[:500]) # 500 premiers caractères suffisent, plus rapide
    except Exception:
        return "unknown"

df["language"] = df["text"].apply(detect_lang_safe)
print(df.groupby(["type", "language"]).size())
```

**Niveau 2 — Analyse de robustesse par langue (fortement recommandé, réutilise le code de la section 7).**

Comme pour `type` et `tactic`, calcule le F1-score séparément par langue :

```
for name, model in best_models.items():
    df_full[f"pred_{name}"] = model.predict(X_full)

for lang in df_full["language"].value_counts().head(3).index:
    subset = df_full[df_full["language"] == lang]
    if subset["label"].nunique() > 1:
        f1_lang = f1_score(subset["label"], subset[f"pred_{best_name}"], zero_division=0)
        print(f"Langue '{lang}': F1 = {f1_lang:.3f} (n={len(subset)})")
```

Si le score chute nettement sur le français, c’est une observation scientifique honnête à documenter — pas un échec, un résultat en soi (“le modèle, entraîné majoritairement sur des données anglaises, généralise moins bien au français, ce qui ouvre une piste d’amélioration”).

**Niveau 3 — Rééquilibrage actif (optionnel, si tu as le temps, très valorisant pour le jury).** Deux options gratuites pour obtenir un vrai bloc “Ham” en français et permettre une comparaison 100% française :

- **Option A — Traduction automatique locale et gratuite** d’un sous-ensemble d’Enron/Nazario vers le français, via un modèle de traduction open source (aucune clé API, tourne en local) :

```

from transformers import MarianMTModel, MarianTokenizer # inclus dans transformers, déjà installé

model_name = "Helsinki-NLP/opus-mt-en-fr"
tokenizer = MarianTokenizer.from_pretrained(model_name)
translation_model = MarianMTModel.from_pretrained(model_name)

def translate_to_french(text, max_len=400):
    text = str(text)[:max_len]
    batch = tokenizer([text], return_tensors="pt", padding=True, truncation=True)
    gen = translation_model.generate(**batch)
    return tokenizer.decode(gen[0], skip_special_tokens=True)

# Traduis un échantillon (500-1000 lignes suffisent, la traduction complète serait longue)
df_sample = df_source.sample(n=800, random_state=42)
df_sample["text"] = df_sample["text"].apply(translate_to_french)
df_sample["language"] = "fr_translated"
df_sample.to_csv("../data/raw/enron_nazario_fr.csv", index=False)

```

- **Option B — Génération d’un petit bloc “Ham en français” via ton LLM**, en réutilisant le script de la section 3.3 avec un prompt inversé (demander des emails professionnels légitimes plutôt que du phishing) — rapide à mettre en place puisque tu as déjà l’infrastructure Ollama :

```

def generate_legit_email(profile):
    prompt = f"""Rédige un email professionnel légitime en français, envoyé à {profile['nom']}, {profile['poste']}, concernant {profile['contexte']}. Ton neutre, factuel, sans aucune tentative de manipulation ni lien suspect. Réponds STRICTEMENT : OBJET: <objet> CORPS: <corps>"""
    response = ollama.chat(model='llama3', messages=[{'role': 'user', 'content': prompt}])
    return response['message']['content']

```

Avec ce bloc, tu peux construire un **sous-dataset 100% français** (ham légitime français + phishing IA français) pour une comparaison propre, en complément du dataset hybride principal anglais/français mixte — les deux analyses cohabitent dans le rapport et montrent une vraie maîtrise méthodologique.

**Ce que tu dois retenir pour le rapport, quel que soit le niveau choisi :** ne jamais présenter un score global sans avoir vérifié qu’il n’est pas biaisé par la langue. Même le Niveau 1 (juste la colonne `language` + un tableau croisé) suffit à anticiper la critique d’un jury sur ce point — les Niveaux 2 et 3 transforment la limite en argument de rigueur scientifique.

## 4.4 Nettoyage Enron/Nazario

```

import pandas as pd

df_raw = pd.read_csv("../data/raw/phishing_email_dataset_kaggle.csv")
print(df_raw.columns.tolist()) # vérifie toujours les noms réels avant de coder en dur

df_clean = df_raw.rename(columns={"text_combined": "text"})[["text", "label"]].dropna()
df_clean["type"] = df_clean["label"].map({0: "enron_ham", 1: "nazario_real_phishing"})
df_clean.to_csv("../data/raw/enron_nazario_clean.csv", index=False)
print(df_clean["label"].value_counts())

```

## 4.5 Script ETL — fusion du dataset hybride final

Crée `membre_B_data_ia/preprocessing/build_hybrid_dataset.py` :

```

import pandas as pd

TARGET_TOTAL = 1000 # ajuste selon le volume total visé
RATIO_HAM, RATIO_PHISH_REAL, RATIO_PHISH_AI = 0.30, 0.40, 0.30

df_source = pd.read_csv("../data/raw/enron_nazario_clean.csv")
df_ham = df_source[df_source["label"] == 0]
df_phish_real = df_source[df_source["label"] == 1]

# Réutilise tes propres emails générés en section 3, reformatés pour l'entraînement
df_ai_raw = pd.read_csv("../data/raw/ai_generated_phishing.csv")
df_ai = pd.DataFrame({
    "text": df_ai_raw["subject"] + "\n" + df_ai_raw["body"],
    "label": 1,
    "type": "ai_phishing_2026",
    "tactic": df_ai_raw["tactic"], # conservée pour l'analyse de robustesse par tactique
})

n_ham = int(TARGET_TOTAL * RATIO_HAM)
n_phish_real = int(TARGET_TOTAL * RATIO_PHISH_REAL)
n_ai = min(len(df_ai), int(TARGET_TOTAL * RATIO_PHISH_AI))

df_ham_sample = df_ham.sample(n=min(n_ham, len(df_ham)), random_state=42)
df_phish_real_sample = df_phish_real.sample(n=min(n_phish_real, len(df_phish_real)), random_state=42)
df_ai_sample = df_ai.sample(n=n_ai, random_state=42) if len(df_ai) > n_ai else df_ai

final_dataset = pd.concat([df_ham_sample, df_phish_real_sample, df_ai_sample], ignore_index=True)
final_dataset = final_dataset.sample(frac=1, random_state=42).reset_index(drop=True)
final_dataset.to_csv("../data/processed/dataset_hybride_phishing_2026.csv", index=False)

print("✓ Dataset hybride final :)")
print(final_dataset["type"].value_counts())

```

**Anticipation :** si tu n’as produit “que” 60 emails IA alors que 30% du total en demanderait 300, le script s’adapte automatiquement. Documente le ratio réel obtenu dans le rapport plutôt que de le cacher — c’est une observation méthodologique honnête, pas une faiblesse.

## 5. Prétraitement NLP

```

import nltk
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
import re

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english')) | set(stopwords.words('french'))

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', ' URLTOKEN ', text)
    text = re.sub(r'\S@\S+', ' EMAILTOKEN ', text)
    text = re.sub(r'[^\w-zâäåæéëîïôóùüÿç\s]', ' ', text)
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens if t not in stop_words and len(t) > 2]
    return " ".join(tokens)

df = pd.read_csv("../data/processed/dataset_hybride_phishing_2026.csv")
df['clean_text'] = df['text'].apply(clean_text)
df.to_csv("../data/processed/dataset_hybride_preprocessed.csv", index=False)

```

Garder `URLTOKEN` / `EMAILTOKEN` comme tokens normalisés (plutôt que de tout supprimer) préserve un signal discriminant utile : la présence d’un lien ou d’une adresse est souvent elle-même informative.

## 6. Feature engineering + entraînement avec validation croisée

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score, classification_report

vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
X = vectorizer.fit_transform(df['clean_text'])
y = df['label']

X_train, X_test, y_train, y_test, type_train, type_test = train_test_split(
    X, y, df['type'], test_size=0.2, random_state=42, stratify=y
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
param_grids = {
    "Naive Bayes": (MultinomialNB(), {"alpha": [0.1, 0.5, 1.0]}),
    "Random Forest": (RandomForestClassifier(random_state=42), {"n_estimators": [100, 200], "max_depth": [None, 20]}),
    "Logistic Regression": (LogisticRegression(max_iter=1000), {"C": [0.1, 1.0, 10.0]}),
}

best_models, results = {}, {}
for name, (model, params) in param_grids.items():
    grid = GridSearchCV(model, params, cv=cv, scoring='f1', n_jobs=-1)
    grid.fit(X_train, y_train)
    best_models[name] = grid.best_estimator_
    y_pred = grid.best_estimator_.predict(X_test)
    results[name] = {
        "best_params": grid.best_params_,
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "f1": f1_score(y_test, y_pred),
    }
    print(f"--- {name} ({grid.best_params_}) ---")
    print(classification_report(y_test, y_pred))
```

## 7. Le test décisif : robustesse par type ET par tactique

C'est le **résultat central du projet**, et grâce à la colonne `tactic`, tu peux aller plus loin que la plupart des projets similaires :

```
df_test_analysis = pd.DataFrame({"true": y_test.values, "type": type_test.values})

for name, model in best_models.items():
    df_test_analysis[f"pred_{name}"] = model.predict(X_test)

# Robustesse par type de donnée (ham / phishing réel / phishing IA)
for name in best_models:
    print(f"\n=== {name} - F1 par type ===")
    for t in df_test_analysis["type"].unique():
        subset = df_test_analysis[df_test_analysis["type"] == t]
        if subset["true"].nunique() > 0:
            f1_t = f1_score(subset["true"], subset[f"pred_{name}"], zero_division=0)
            print(f"    {t}: F1 = {f1_t:.3f} (n={len(subset)})")
```

Pour la robustesse **par tactique psychologique** (recroise avec le dataset complet, pas seulement le split test, pour avoir assez de données par tactique) :

```

df_full = df.copy()
X_full = vectorizer.transform(df_full['clean_text'])

for name, model in best_models.items():
    df_full[f"pred_{name}"] = model.predict(X_full)

df_ai_only = df_full[df_full["type"] == "ai_phishing_2026"]
print("\n=== Détectabilité par tactique psychologique (meilleur modèle) ===")
best_name = max(results, key=lambda n: results[n]["f1"])
for tactic in df_ai_only["tactic"].dropna().unique():
    subset = df_ai_only[df_ai_only["tactic"] == tactic]
    taux_detection = (subset[f"pred_{best_name}"] == 1).mean() * 100
    print(f"    {tactic}: {taux_detection:.1f}% détectés (n={len(subset)})")

```

**C’est ce tableau qui alimente la synthèse croisée avec le Membre A** : si la tactique “urgence” a le taux de clic humain le plus élevé (ses stats GoPhish) ET le taux de détection IA le plus bas (tes stats), vous avez une conclusion forte et originale pour le rapport — *“la tactique qui trompe le plus les humains est aussi celle qui trompe le plus l’IA de détection”*.

## 8. Explicabilité (SHAP)

```

import shap
import matplotlib.pyplot as plt

explainer = shap.LinearExplainer(best_models["Logistic Regression"], X_train, feature_names=vectorizer.get_feature_names_out())
shap_values = explainer.shap_values(X_test[:100])
shap.summary_plot(shap_values, X_test[:100], feature_names=vectorizer.get_feature_names_out(), show=False)
plt.tight_layout()
plt.savefig("../explainability/shap_summary.png", dpi=200)

```

Montre les mots qui poussent le plus vers “phishing” (ex: “urgent”, “cliquez”, “vérifier”) — un visuel très parlant en soutenance.

## 9. Dashboards de performance

```

import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from sklearn.metrics import confusion_matrix

for name, model in best_models.items():
    cm = confusion_matrix(y_test, model.predict(X_test))
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Sain', 'Phishing'], yticklabels=['Sain', 'Phishing'])
    plt.title(f"Matrice de confusion - {name}")
    plt.tight_layout()
    plt.savefig(f"../dashboard_app/confusion_matrix_{name.replace(' ', '_')}.png", dpi=200)
    plt.close()

df_results = pd.DataFrame(results).T
df_results[['precision', 'recall', 'f1']].astype(float).plot(kind='bar', figsize=(8, 5))
plt.title("Comparaison des métriques par modèle")
plt.tight_layout()
plt.savefig("../dashboard_app/comparaison_modeles.png", dpi=200)
plt.close()

for label_val, name in [(0, 'sain'), (1, 'phishing')]:
    text_blob = " ".join(df[df['label'] == label_val]['clean_text'])
    wc = WordCloud(width=800, height=400, background_color='white').generate(text_blob)
    plt.figure(figsize=(10, 5))
    plt.imshow(wc, interpolation='bilinear')
    plt.axis('off')
    plt.savefig(f"../dashboard_app/wordcloud_{name}.png", dpi=200)
    plt.close()

print("✓ Dashboards générés.")

```

Utilise la même palette de couleurs que le Membre A pour ses graphiques comportementaux (accordez-vous sur #4C72B0 / #C44E52 par exemple) — le rapport final doit avoir l’air d’un tout cohérent.

## 10. Bonus fort impact : démo Streamlit pour la soutenance

```

# Sauvegarde après entraînement :
# joblib.dump(best_models["Logistic Regression"], "model_final.pkl")
# joblib.dump(vectorizer, "vectorizer_final.pkl")

import streamlit as st
import joblib

model = joblib.load("model_final.pkl")
vectorizer = joblib.load("vectorizer_final.pkl")

st.title("🔍 Détecteur de Phishing IA - Démo PFE 2026")
email_input = st.text_area("Colle un email à analyser", height=200)

if st.button("Analyser"):
    if email_input.strip():
        cleaned = clean_text(email_input)
        X_input = vectorizer.transform([cleaned])
        pred = model.predict(X_input)[0]
        proba = model.predict_proba(X_input)[0]
        if pred == 1:
            st.error(f"⚠️ PHISHING détecté (confiance : {proba[1]*100:.1f}%)")
        else:
            st.success(f"✓ Email légitime (confiance : {proba[0]*100:.1f}%)")

```

```
streamlit run membre_B_data_ia/dashboard_app/demo_streamlit.py
```

Pour un lien public gratuit : [share.streamlit.io](https://share.streamlit.io), connecte GitHub, sélectionne le repo et le fichier.

## 11. Travail commun avec le Membre A

---

## Semaine 2 — Livraison des emails IA

Livre `ai_generated_phishing.csv` au Membre A au plus tôt. Il les injecte dans GoPhish et lance sa campagne.

## Semaine 3 — Fusion des graphiques + synthèse croisée

Tes courbes de performance (F1 par type, par tactique, SHAP) rejoignent ses statistiques comportementales (taux de clic, délai de réaction, par tactique). Prépare en particulier le tableau croisé tactique ↔ taux de clic ↔ taux de détection décrit en section 7 — c'est votre résultat le plus original.

---

### 12. Livrables attendus (Membre B)

- ☐ `ai_generated_phishing.csv` livré au Membre A (50-60 emails, 5 tactiques diversifiées)
  - ☐ `dataset_hybride_phishing_2026.csv` (30/40/30 respecté au mieux du possible)
  - ☐ Pipeline de prétraitement NLP documenté
  - ☐ Au moins 3 modèles entraînés avec validation croisée + GridSearch (BERT en bonus)
  - ☐ Analyse de robustesse par type ET par tactique psychologique
  - ☐ Graphique SHAP d'explicabilité
  - ☐ Dashboard complet (matrices de confusion, comparaison de modèles, word clouds)
  - ☐ Démo Streamlit (bonus fort impact)
  - ☐ Tableau croisé final avec les données comportementales du Membre A
- 

### 13. Points de vigilance / anticipation des problèmes

- **Livre tes emails IA en premier, avant de te lancer dans le NLP** — le Membre A a besoin de temps pour construire ses templates GoPhish et lancer sa campagne avant la semaine 3.
  - **Vérifie toujours les noms de colonnes réels** du CSV Kaggle/HF téléchargé avant de coder en dur.
  - **Déséquilibre des classes** : `stratify=y` obligatoire ; envisage `imbalanced-learn` / `SMOTE` si le déséquilibre persiste après échantillonnage.
  - **Barrière de la langue (Français vs Anglais)** : Enron/Nazario en anglais, tes emails IA en français — ne mélange jamais sans traçabilité. Applique au minimum le Niveau 1 de la section 4.3 (colonne `language` + tableau croisé type/langue) avant tout entraînement, pour ne jamais présenter un score global qui cacherait un biais de langue.
  - **Ne renomme jamais** les colonnes `text` / `label` / `type` / `tactic` sans en discuter avec le Membre A, ça casserait sa capacité à croiser vos résultats.
  - **Format CSV robuste** : `quoting=1` avec pandas pour éviter qu'un email contenant une virgule ne casse le fichier — l'erreur n°1 qui bloquerait l'intégration.
  - **Ne bloque pas sur IWSPA-AP** : Enron + Nazario suffisent largement.
  - **Sauvegardes** : commit régulièrement, messages clairs.
- 

### 14. Calendrier suggéré

| Semaine | Toi (Membre B)                                                                                                                                            |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | Setup (Ollama + Python), script de génération, production des 50-60 emails IA → livraison <code>ai_generated_phishing.csv</code> au Membre A dès que prêt |
| 2       | Récupération Enron/Nazario, nettoyage, fusion ETL du dataset hybride                                                                                      |
| 3       | Prétraitement, entraînement + GridSearch, analyse de robustesse par type/tactique, réception des stats GoPhish du Membre A                                |
| 4       | SHAP, dashboards complets, tableau croisé tactique/clic/détection                                                                                         |
| 5       | Démo Streamlit, finalisation des visuels                                                                                                                  |
| 6+      | Rédaction méthodologique, support à la synthèse finale et à la soutenance                                                                                 |